# Spinlocks vs. mutexes: review and Cortex-A9 benchmarks

Nov 5, 2026 · @Brian

## Summary

Use `std::mutex` by default; use a spinlock only for very short, hot critical sections with no more contending threads than cores. On the dual Cortex-A9 target, the reworked spinlocks are 2–4× faster than mutexes on short critical sections, but the reworked `CSpinlock` can starve a thread when threads re-lock back to back.

- **`CRecursiveSpinlock`**: adopt the reworked version. It is 2× faster than the current one on short sections, 3.4× faster when re-entered, and fair.
- **`CSpinlock`**: the rework is only 10–30% faster than the current lock, and gave one thread 3–9% of its fair share in the worst-case test. A candidate fix (`nobarge32`) removes the starvation at about 2.5× the cost in that worst case, still faster than `std::mutex`.
- **Decision needed**: make `nobarge32` the new `CSpinlock`, or keep the current rework.

Code: `github.com/bbiggs4vt/locks`, branch `main`.

## Background

The review started from two Boost-based locks already in use:

- **`CSpinlock`**: a test-and-set spinlock. Each failed attempt does an atomic exchange, then calls `boost::this_thread::yield()`.
- **`CRecursiveSpinlock`**: a recursive spinlock based on Preshing's recursive mutex, using an owner id, a recursion count and a counter of contending threads.

In the benchmark, `v1::` is the current code. `v1::CSpinlock` is unchanged. `v1::CRecursiveSpinlock` has one change, the race fix below: without it the recursive lock lets two threads in at once, and the benchmark stops when it detects lost updates.

## Issues found and fixes

The most serious issue: the original `CRecursiveSpinlock` could let two threads hold the lock at once. New tests caught it as assertion failures and hangs.

| Issue | Lock | Effect | Fix |
|---|---|---|---|
| Uncontended path claims ownership with a plain store after `++mCounter` | `CRecursiveSpinlock` | A second thread can CAS itself in during the gap; both enter the lock | Claim ownership with a CAS; drop `mCounter` |
| `scoped_lock` does not track ownership | Both | Calling `unlock()` early, then the destructor, can release another thread's lock | Guard tracks ownership (`owns_lock()`) |
| No `try_lock()` | Both | Cannot be used with `std::lock` or `std::unique_lock` try/defer modes | Added `try_lock()` |
| Misuse caught only by `assert` | `CRecursiveSpinlock` | Unlocking from the wrong thread silently corrupts the lock in release builds | `unlock()` by a non-owner calls `std::abort()` |
| Waiters retry the atomic exchange constantly | `CSpinlock` | Waiters fight the holder for the cache line | Waiters spin on a plain read first (test-and-test-and-set) |
| `yield()` (a syscall) on every failed attempt | Both | Each retry costs more than the lock | Bounded exponential backoff with a CPU hint, then `yield()` |
| Four seq_cst atomic operations per lock/unlock pair | `CRecursiveSpinlock` | Extra barriers and cache traffic | One acquire CAS, one release store; re-entry uses no atomics |
| Possibly not lock-free `boost::atomic<thread::id>` | `CRecursiveSpinlock` | On some Boost configs a hidden lock sits inside the spinlock | `static_assert` that it is lock-free |
| Lock shares a cache line with nearby data | Both | False sharing | `alignas(64)` |

The spin hint and backoff limit are now a template policy (`BasicSpinlock<SpinPolicy<Hint, MaxBackoff>>`). `CSpinlock` and `CRecursiveSpinlock` are typedefs for each architecture's default. On the Cortex-A9 neither the `isb` hint nor a 16× longer backoff made a measurable difference, so the defaults (`yield`, 64) stay.

## Benchmark method

`lock_benchmark` measures median wall time per lock acquisition over 5 repetitions of 200,000 operations per thread, at 1, 2 and 4 threads on the 2-core target.

| Phase | What each thread does | Measures |
|---|---|---|
| Throughput, `tiny` | Lock, increment a shared counter, unlock | Pure lock overhead (ns per acquisition) |
| Throughput, `medium` | Lock, about 100 dependent multiply-adds, unlock | Cost with a longer critical section |
| Throughput, `nested` | Lock twice, increment, unlock twice (recursive locks only) | Re-entry cost |
| Fairness, `gap=0` | Lock, increment, unlock, repeat immediately for 300 ms | Least-served thread's share of acquisitions (1.00 = even), worst case |
| Fairness, `gap=100` | As above, plus about 100 multiply-adds of thread-local work between acquisitions | The same share, with a realistic pattern |

Every run checks that no updates to the shared counter were lost. The locks compared: the reworked locks, `v1::` (current code), `boost::` and `std::` mutexes, spin-policy variants (`[isb]`, `[backoff1024]`), and three candidate fixes for starvation (`TicketSpinlock`, `CSpinlock[nobarge]`, `CSpinlock[nobarge32]`).

Target: dual Cortex-A9 (ARMv7, CPU part `0xc09`), GCC 13.3, 32-bit build. Atomics there are `ldrex`/`strex` retry loops.

## Throughput on the Cortex-A9

Both spinlocks are about 2× faster than mutexes uncontended and 2–4× faster under contention on short sections. Numbers are median ns per acquisition, lower is better, from the second board run.

| Lock | Workload | 1 thread | 2 threads | 4 threads |
|---|---|---|---|---|
| `CSpinlock` | tiny | 95 | 86 | 100 |
| `CSpinlock[nobarge32]` | tiny | 90 | 215 | 264 |
| `v1::CSpinlock` | tiny | 92 | 104 | 138 |
| `std::mutex` | tiny | 171 | 387 | 420 |
| `CRecursiveSpinlock` | tiny | 92 | 94 | 107 |
| `v1::CRecursiveSpinlock` | tiny | 192 | 204 | 208 |
| `std::recursive_mutex` | tiny | 197 | 421 | 421 |
| `CSpinlock` | medium | 1415 | 1418 | 1709 |
| `CSpinlock[nobarge32]` | medium | 1401 | 1513 | 1968 |
| `v1::CSpinlock` | medium | 1404 | 1431 | 2079 |
| `std::mutex` | medium | 1525 | 2526 | 2827 |
| `CRecursiveSpinlock` | medium | 1418 | 1480 | 1982 |
| `v1::CRecursiveSpinlock` | medium | 1591 | 1569 | 1629 |
| `std::recursive_mutex` | medium | 1534 | 2542 | 2955 |
| `CRecursiveSpinlock` | nested | 99 | 100 | 114 |
| `v1::CRecursiveSpinlock` | nested | 338 | 337 | 339 |
| `std::recursive_mutex` | nested | 266 | 574 | 606 |

- `boost::` mutexes performed within a few percent of the `std::` ones and are left out.
- `TicketSpinlock` was 744 ns at 2 threads and 7,437 ns at 4 threads on `tiny`: when the next thread in line is not running, every waiter waits for it.

- The one case where v1 wins: medium sections with 4 threads on 2 cores, where `v1::CRecursiveSpinlock` gives up the CPU immediately and lets a preempted holder finish (1,629 vs 1,982 ns, in both board runs).

## Fairness and starvation

The reworked `CSpinlock` starves a waiting thread when threads re-lock back to back; with any work between acquisitions every lock is fair.




lock_benchmark on dual Cortex-A9, fairness phase gap=0, 300 ms per run

- **Likely cause:** the thread that unlocks re-locks a few nanoseconds later while it still has the lock's cache line. A waiter only reads the lock, so by the time it sees it free and tries the exchange, the lock is taken again. v1's waiters attempt the exchange constantly, so they catch the release. The effect did not reproduce on x86.
- **With a realistic gap** ( `gap=100` ), every lock scored 0.84–1.00, and all delivered about 1.1 Mops/s. The lock was free most of the time, so the starvation only matters for code that takes the same lock in a tight loop.
- **Spin-policy variants** ( `[isb]` , `[backoff1024]` ) did not fix it: 0.02–0.42.

The candidate fixes:

| Candidate | How it works | Fairness, gap=0 (2 / 4 threads) | Cost vs `CSpinlock` , tiny (2 / 4 threads) | Verdict |
|---|---|---|---|---|
| `CSpinlock[nobarge32]` | A thread arriving while others wait first gives them 32 hint calls to take the lock | 0.89 / 0.94 | 2.5× / 2.6× | Best option |
| `CSpinlock[nobarge]` | The same, with a 256-call grace period | 0.79 / 0.80 | 2.7× / 2.7×; 1.8× slower on medium | Rejected |
| `TicketSpinlock` | Strict first-come, first-served | 0.65 / 0.31 | 8.7× / 74× | Rejected |

## When to use a spinlock vs a mutex

Default to `std::mutex` ; switch to a spinlock only where you have measured that the mutex costs something and all four conditions below hold.

Why the mutex is the default:

- **Under light contention both perform the same.** In the `gap=100` run every lock delivered about 1.1 Mops/s and was fair.
- **A mutex fails gracefully.** If the holder is preempted or stalls, waiters sleep instead of burning the other core, which on a 2-core board is half the CPU.
- **A mutex handles more threads than cores,** works with priority inheritance, and is understood by tools such as ThreadSanitizer.
- **Spinlocks have pattern-specific risks,** such as the starvation measured at `gap=0` .

Use a spinlock only when all of these hold:

1. **The critical section is very short and bounded:** a few loads and stores, with no system calls, allocation, I/O, logging or other locks inside. On the A9 the lock costs about 90 ns against about 170 ns for `std::mutex` uncontended.

2. **Contending threads are no more than cores,** ideally pinned or at real-time priority. On this board that means at most 2 threads using the lock.
3. **The lock is hot enough to appear in a profile.** Saving 80 ns matters at 100,000 acquisitions a second, not at 100.
4. **Sleeping is not acceptable,** for example in a real-time loop. Contended mutex waits go through the kernel: 380–420 ns on the A9, against 90–100 ns for the spinlock.

The same rules apply to the recursive lock. A recursive lock is often a sign the locking could be simpler; if re-entry can be designed out, do it.

Signs a spinlock should go back to a mutex:

- CPU usage rises without a rise in throughput.
- Latency spikes when the system is busy.
- One thread makes little progress.
- Something slow is added inside the critical section. Write the constraint next to the lock.

## Next steps and reproducing the results

The recommendation is to make `nobarge32` the new `CSpinlock`: it costs nothing uncontended, stays faster than `std::mutex` everywhere, and removes the starvation. Keeping the current rework is defensible only if no code re-locks the same lock back to back.

- [ ] Decide: `nobarge32` as `CSpinlock`, or keep the current rework
- [ ] If `nobarge32`: move it into `include/locks/spinlock.hpp` with the grace period as a policy parameter, and remove `TicketSpinlock` and the 256 variant from the benchmark
- [ ] Check each existing use of `CSpinlock` / `CRecursiveSpinlock` against the four conditions above; move those that fail to `std::mutex`
- [ ] Adopt the reworked `CRecursiveSpinlock`

To reproduce on the target:

```
cmake -S . -B build -DLOCKS_NATIVE_CPU=ON
cmake --build build -j
ctest --test-dir build
./build/lock_benchmark | tee results.txt
```

- **Options:** `--ops`, `--reps`, `--threads`, `--fairness-ms` and `--csv`.
- **Build-info line:** the first output line records the architecture, compiler, atomics, hint and backoff limit; keep it with any results.
- **Files:** the current code is `bench/spinlock_v1.hpp`, and the candidates are in `bench/spinlock_candidates.hpp`.